Minute Bars from Tick Data

Theory, Construction, and Implementation from NASDAQ ITCH

IEOR E4733 — Algorithmic Trading — Spring 2026 — Week 7

Contents

1 What Is a Bar? 1

2 Standard OHLCV Metrics 1

3 Extended Metrics 1
- 3.1 VWAP — Volume-Weighted Average Price . . . 1
- 3.2 Dollar Volume . . . 1
- 3.3 Trade Count . . . 2
- 3.4 Shares at Bid and Ask . . . 2

4 Trade Direction: Classification Methods 2
- 4.1 Order Book Join (Primary) . . . 2
- 4.2 Tick Rule (Fallback) . . . 2

5 Data Source: NASDAQ ITCH 2
- 5.1 Relevant Message Types . . . 3
- 5.2 Key Data Detail: `stock_locate` . . . 3

6 Implementation: Minute Bar Pipeline 3
- 6.1 Step 1 — Resolve Symbol to `stock_locate` . . . 3
- 6.2 Step 2 — Load Executed Trades . . . 3
- 6.3 Step 3 — Apply Tick Rule . . . 3
- 6.4 Step 4 — Resample to Minute Bars . . . 4
- 6.5 Step 5 — Persist to HDF5 . . . 4

7 Output Schema 4

8 Summary 4

1 What Is a Bar?

A **bar** is a summary record that compresses a sequence of raw trades (ticks) into a fixed set of statistics over a defined interval. The **minute bar** is the most common unit in intraday algorithmic trading: it aggregates all trades that occurred within a single calendar minute into a small, structured record.

> A minute bar answers: *where did the price start, where did it go, and how much changed hands during this minute?*

Bars serve several purposes:

- Reduce the dimensionality of tick data (millions of events → hundreds of bars).
- Provide a regular time grid suitable for time-series models and vectorized computation.
- Expose standard metrics (OHLCV) that are inputs to virtually every intraday signal.

2 Standard OHLCV Metrics

Every bar contains five canonical fields:

Field	Symbol	Definition
Open	O_t	Price of the **first** trade in the minute
High	H_t	**Maximum** trade price in the minute
Low	L_t	**Minimum** trade price in the minute
Close	C_t	Price of the **last** trade in the minute
Volume	V_t	Total **shares** traded in the minute: $V_t = \sum_i s_i$

3 Extended Metrics

Beyond OHLCV, several additional statistics are routinely computed per bar.

3.1 VWAP — Volume-Weighted Average Price

The **VWAP** is the average price paid per share within the bar, weighted by trade size:

$$\text{VWAP}_t = \frac{\sum_i p_i \cdot s_i}{\sum_i s_i} = \frac{\text{Dollar Volume}_t}{V_t}$$

where p_i is the price and s_i the size of the i-th trade in the minute.

VWAP is the standard benchmark for institutional execution quality: a buy order executed below VWAP is considered favorable; one executed above is costly.

3.2 Dollar Volume

$$\text{DV}_t = \sum_i p_i \cdot s_i$$

Dollar volume (notional value traded) is preferred over share volume for cross-sectional comparisons, since it normalizes out differences in share price across stocks.

3.3 Trade Count

$$N_t = \text{number of individual trades in the minute}$$

Trade count is an independent proxy for activity and liquidity. High N_t with low V_t signals fragmented, small-lot trading (often algorithmic); low N_t with high V_t signals large block trades (often institutional).

3.4 Shares at Bid and Ask

Each trade can be classified as **buyer-initiated** (executed at the ask) or **seller-initiated** (executed at the bid):

$$n_t^{\text{ask}} = \sum_{i:\,\delta_i=+1} s_i, \qquad n_t^{\text{bid}} = \sum_{i:\,\delta_i=-1} s_i$$

where $\delta_i = +1$ for a buyer-initiated trade and -1 for a seller-initiated trade. The **order flow imbalance** per bar is:

$$\text{OFI}_t = n_t^{\text{ask}} - n_t^{\text{bid}}$$

A persistently positive OFI indicates buying pressure; negative indicates selling pressure. OFI is one of the most predictive short-horizon signals in the microstructure literature.

4 Trade Direction: Classification Methods

Trade direction δ_i is not always directly observable. For NASDAQ ITCH data, we recover it from two sources:

4.1 Order Book Join (Primary)

ITCH message types:

- **A** / **F** (Add Order): records the `buy_sell_indicator` of each limit order placed in the book.
- **E** (Order Executed) and **C** (Executed at Different Price): record the execution event via `order_reference_number`.

By joining E/C messages to A/F messages on `order_reference_number`, we recover the original limit order's direction and assign it to the execution. Since limit orders resting in the book are *passive*, the *contra* side (the aggressor) takes that direction:

Resting order	Aggressor fills at	Trade direction δ_i
Sell limit (ask)	Ask price	$+1$ (buyer-initiated)
Buy limit (bid)	Bid price	-1 (seller-initiated)

4.2 Tick Rule (Fallback)

For **P** (non-cross trade) messages where direction is unknown, the **tick rule** is applied:

$$\delta_i = \begin{cases} +1 & \text{if } p_i \geq p_{i-1} \quad \text{(up-tick or zero-tick: buyer-initiated)} \\ -1 & \text{if } p_i < p_{i-1} \quad \text{(down-tick: seller-initiated)} \end{cases}$$

The tick rule is a simple, robust heuristic due to Lee and Ready (1991). It performs well in liquid markets where direction can usually be inferred from price movement, but it misclassifies at inflection points and during rapid quote changes.

5 Data Source: NASDAQ ITCH

NASDAQ TotalView-ITCH is the native binary feed from NASDAQ that delivers the complete order book in real time. A single day's feed contains every order add, cancel, modify, and execution for all NASDAQ-listed securities — typically > 200 million messages per day.

5.1 Relevant Message Types

Type	Name	Purpose
`A`	Add Order (no MPID)	New limit order; contains symbol, side, price, size
`F`	Add Order (with MPID)	Same as A, with market participant identifier
`E`	Order Executed	Full or partial fill at the limit price
`C`	Order Executed (new price)	Fill at a different price (e.g. price improvement)
`P`	Trade (non-cross)	Standalone trade; direction not linked to an order
`R`	Stock Directory	Maps integer `stock_locate` codes to ticker symbols

5.2 Key Data Detail: `stock_locate`

ITCH does not store ticker symbols in trade/order messages. Instead, each message carries an integer `stock_locate` code. The mapping from code to symbol is in the `R` (Stock Directory) table:

```
with pd.HDFStore("itch.h5", mode="r") as store:
    R = store["R"][["stock_locate", "stock"]]

locate = int(R[R["stock"] == "AAPL"]["stock_locate"].iloc[0])  # e.g. 13
```

All A, F, E, C, P messages must be filtered using this code, not a string symbol.

6 Implementation: Minute Bar Pipeline

6.1 Step 1 — Resolve Symbol to `stock_locate`

```
def get_locate(symbol, h5path):
    with pd.HDFStore(h5path, mode="r") as store:
        R = store["R"][["stock_locate", "stock"]]
    return int(R[R["stock"] == symbol]["stock_locate"].iloc[0])
```

6.2 Step 2 — Load Executed Trades

```
PRICE_SCALE     = 10_000          # ITCH stores price * 10,000
MARKET_OPEN_NS  = 9 * 3_600 * 1_000_000_000 + 30 * 60 * 1_000_000_000
MARKET_CLOSE_NS = 16 * 3_600 * 1_000_000_000

def load_trades(symbol, h5path):
    locate = get_locate(symbol, h5path)
    orders = load_orders(locate, h5path)   # A + F messages for direction
    frames = []
```

```
    # E: Order Executed (direction from order book join)
    with pd.HDFStore(h5path, mode="r") as store:
        e = store.select("E", where=f"stock_locate={locate}")
    merged = e.rename(columns={"executed_shares": "shares"}).merge(
        orders[["order_reference_number", "buy_sell_indicator", "price"]],
        on="order_reference_number", how="inner")
    merged["price"] /= PRICE_SCALE
    frames.append(merged[["timestamp", "price", "shares", "buy_sell_indicator"]])

    # P: Non-cross trades (direction unknown -> tick rule later)
    with pd.HDFStore(h5path, mode="r") as store:
        p = store.select("P", where=f"stock_locate={locate}")
    p["price"] /= PRICE_SCALE
    p["buy_sell_indicator"] = 0
    frames.append(p[["timestamp", "price", "shares", "buy_sell_indicator"]])

    df = pd.concat(frames)
    df["timestamp"] = df["timestamp"].dt.total_seconds().mul(1e9).astype("int64")
    return df[(df.timestamp >= MARKET_OPEN_NS) & (df.timestamp < MARKET_CLOSE_NS)]
```

6.3 Step 3 — Apply Tick Rule

```
def apply_tick_rule(trades):
    mask = trades["buy_sell_indicator"] == 0
    diff = trades["price"].diff()
    trades.loc[mask, "buy_sell_indicator"] = np.where(diff[mask] >= 0, 1, -1)
    return trades
```

6.4 Step 4 — Resample to Minute Bars

```
def build_minute_bars(trades, date="2019-10-30"):
    trades = apply_tick_rule(trades)
    trades["dt"] = pd.Timestamp(date) + pd.to_timedelta(trades["timestamp"], unit="ns")
    t = trades.set_index("dt").sort_index()

    t["dv"]     = t["price"] * t["shares"]           # dollar value per trade
    t["at_ask"] = t["shares"].where(t["buy_sell_indicator"] ==  1, 0)
    t["at_bid"] = t["shares"].where(t["buy_sell_indicator"] == -1, 0)

    r = t.resample("1min")
    bars = pd.DataFrame({
        "open":          r["price"].first(),
        "high":          r["price"].max(),
        "low":           r["price"].min(),
        "close":         r["price"].last(),
        "volume":        r["shares"].sum(),
        "trade_count":   r["price"].count(),
        "dollar_volume": r["dv"].sum(),
        "n_at_ask":      r["at_ask"].sum(),
        "n_at_bid":      r["at_bid"].sum(),
    }).dropna(subset=["open"])

    bars["vwap"] = bars["dollar_volume"] / bars["volume"]
    return bars
```

6.5 Step 5 — Persist to HDF5

```
with pd.HDFStore("minute_bars.h5", mode="w", complevel=5, complib="blosc") as store:
    for symbol, bars in results.items():
        store.put(f"minute_bars/{symbol}", bars, format="table", data_columns=True)
```

7 Output Schema

Each minute bar (one row per minute, one table per symbol) contains:

Column	Type	Description
`open`	float	First trade price in the minute
`high`	float	Highest trade price
`low`	float	Lowest trade price
`close`	float	Last trade price
`volume`	int	Total shares traded
`vwap`	float	Volume-weighted average price
`trade_count`	int	Number of individual trades
`dollar_volume`	float	Notional value ($\sum p_i s_i$)
`n_at_ask`	int	Buyer-initiated shares
`n_at_bid`	int	Seller-initiated shares

Sample output (AAPL, October 30, 2019, first three bars):

minute	open	high	low	close	volume	vwap	trades	n_ask	n_bid
09:30	244.83	245.29	244.66	244.84	78,731	244.93	714	40,534	38,197
09:31	244.81	244.81	244.50	244.77	31,167	244.68	361	17,150	14,017
09:32	244.74	245.15	244.70	244.90	28,473	244.97	414	17,230	11,243

8 Summary

Key takeaways:

1. Minute bars compress raw ITCH tick data into a structured, regular time series.
2. OHLCV captures price range and activity; VWAP and dollar volume normalize across price levels; trade count distinguishes fragmented vs. block flow.
3. n_{ask} and n_{bid} expose the signed order flow imbalance (OFI), one of the strongest predictors of short-horizon price moves.
4. Direction is recovered via order book join (E/C $\rightarrow$ A/F) where available, and the tick rule as a fallback for P messages.
5. ITCH uses integer `stock_locate` codes, not ticker strings — always resolve via the `R` table before querying other message types.